Лабораторная работа № 2

Генетический алгоритм для дискретной задачи с ограничениями

(распределение заявок между исполнителями)

Панов Артемий, Б24-527

2026 г.

1 Постановка задачи и индивидуальный вариант

Требуется решить задачу распределения заявок между исполнителями с различной производительностью при ограничении ёмкости каждого исполнителя генетическим алгоритмом с целочисленным кодированием.

Индивидуальные параметры: $N = 18$, $Q = 2$ (группа 527), $Y = 26$. Номер варианта:

$$V = 1 + \big((17N + 7Q + Y) \bmod 20\big) = 1 + (346 \bmod 20) = 7.$$

Вариант 7 — распределение ограниченного числа заявок между исполнителями с различной производительностью; основное представление — целочисленный вектор.

2 Формальная модель и данные

Есть $N = 30$ заявок с трудоёмкостями $w_i > 0$ и $M = 6$ исполнителей с производительностью p_j (объём в час) и ёмкостью c_j (максимум заявок). Назначение $x_i \in \{1, \dots, M\}$ для каждой заявки; время обработки заявки i исполнителем j равно $t_{ij} = w_i/p_j$. Цель — минимизировать максимальное время завершения (makespan):

$$\min_x \; T(x) = \max_j \sum_{i:\, x_i = j} \frac{w_i}{p_j} \qquad \text{s.t.} \quad \big|\{i : x_i = j\}\big| \le c_j \;\; \forall j.$$

Экземпляр — синтетический, генерируется с фиксированным зерном seed = 746: трудоёмкости $w_i \sim \mathrm{U}(1, 10)$, производительности $p = (0{,}5;\ 0{,}7;\ 1{,}0;\ 1{,}3;\ 1{,}6;\ 2{,}0)$, ёмкости $c = (5;\ 6;\ 7;\ 6;\ 5;\ 6)$, $\sum c_j = 35 \ge 30$ — допустимые решения существуют. Минимальные масштабы по ТЗ выполнены: 30 заявок ≥ 20, 6 исполнителей ≥ 5.

3 Представление особи

Особь — целочисленный вектор $x = (x_1, \dots, x_{30})$, $x_i \in \{0, \dots, 5\}$: элемент i — исполнитель заявки i. Такое кодирование прямо отражает структуру решения (процедура декодирования тривиальна), а генетический материал — это готовые ответы на вопрос «кому выполнять заявку», поэтому рекомбинация наследует осмысленные решения.

4 Генетический алгоритм

Схема работы — на рис. 1, параметры — в таблице 1.

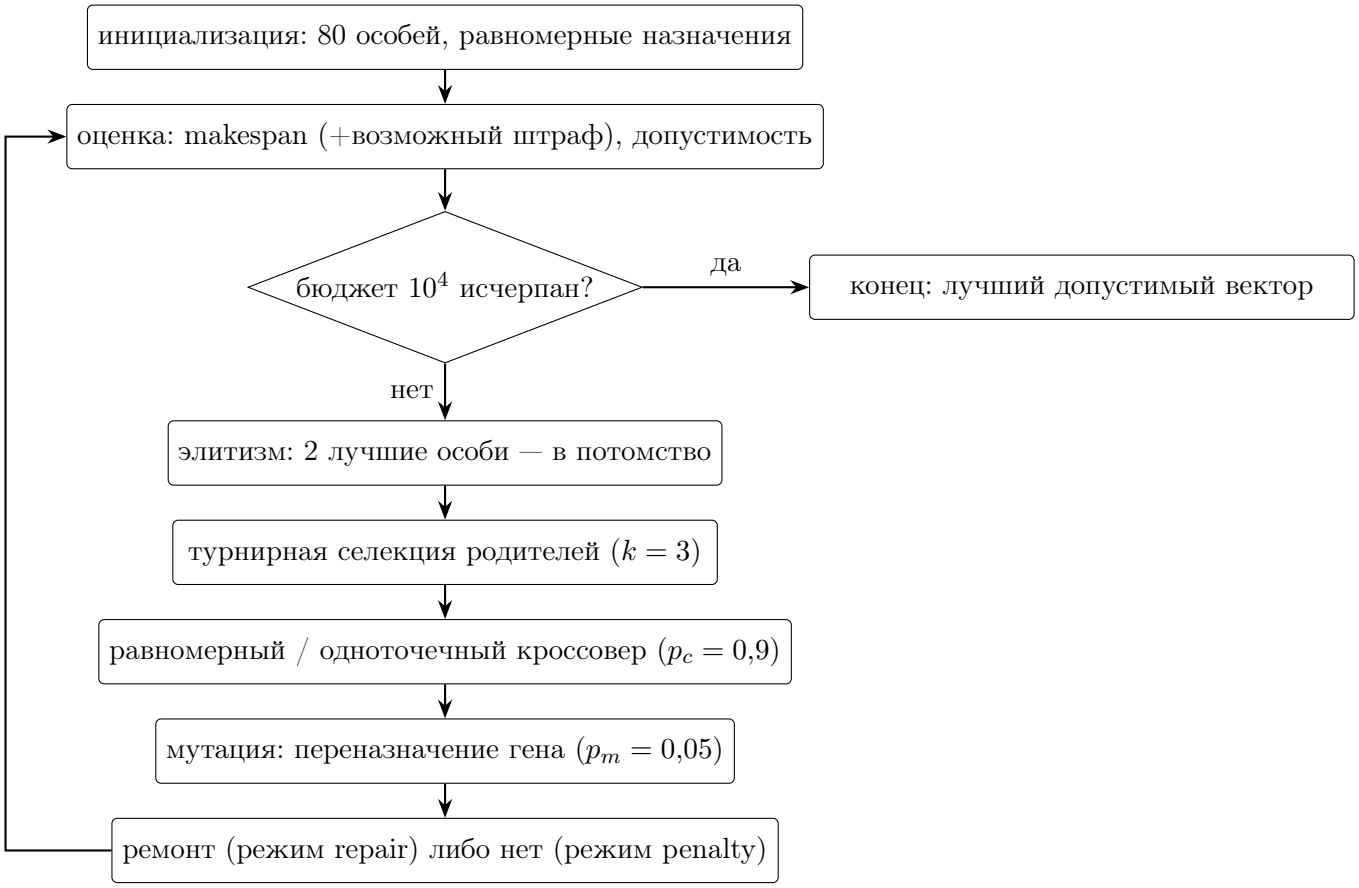


Рис. 1 – Блок-схема генетического алгоритма

Таблица 1 – Параметры алгоритма

Параметр	Значение
Размер популяции	80 (≥ 30 по заданию)
Селекция	турнирная, $k = 3$
Рекомбинация	равномерный либо одноточечный кроссовер, $p_c = 0{,}9$
Мутация	переназначение гена случайному исполнителю, $p_m = 0{,}05$ на ген
Обработка границ	ремонт либо штраф $W \cdot \sum_j \max(0, n_j - c_j)$, $W = 50$
Сохранение лучшего	элитизм, 2 особи
Критерий остановки	бюджет 10 000 вычислений f (инициализация включительно)
Серия	30 независимых запусков, seed $= 8000 + 19i$

Штраф. fitness $= \max_j T_j + W \sum_j \max(0, n_j - c_j)$, $W = 50$: за нарушение ёмкости к значению fitness добавляется большая константа.

Ремонт. Пока есть перегруженный исполнитель, переносится заявка от него к свободному исполнителю, выбираемая по минимуму добавляемого времени w_i/p_k (жадный ремонт).

5 Обоснование выбора операторов

Структура задачи: назначения — множество заявок с естественным целочисленным кодированием. Отсюда:

- **турнирная селекция** ($k = 3$) — умеренное давление, не требующее масштабирования fitness (значения makespan порядка 25–50, но в ранних поколениях штрафные результаты разбегаются на сотни);
- **равномерный и одноточечный кроссоверы** — целочисленный вектор имеет смысловые элементы по заявкам; сравниваются два содержательно разных механизма обмена ответами по заявкам;
- **мутация по гену** — переназначение одной заявки случайному исполнителю, стандартная мелкая варьирующая операция для векторов назначений;
- **ремонт и штраф** — два контрастных способа вернуть решения в допустимое множество; их сравнение показывает, какой из подходов работает лучше.

6 Примеры допустимых и недопустимых решений

Таблица 2 – Проверка допустимости на четырёх решениях

№	счёт по исполнителям	makespan	вердикт
1	(5, 6, 7, 6, 5, 1)	54,842	допустимо
2	(5, 5, 5, 5, 5, 5)	66,730	допустимо
3	(30, 0, 0, 0, 0, 0)	377,898	недопустимо (нарушение 25)
4	(0, 0, 0, 0, 0, 30)	94,475	недопустимо (нарушение 24)

Видно, что допустимые назначения дают makespan порядка 55–67, а грубые нарушения ёмкости дают либо очень плохой makespan, либо резкий штраф, если считать его явно.

7 Методология эксперимента

Три эксперимента, каждый — по 30 независимых запусков (seed $= 8000 + 19i$) при фиксированном бюджете 10 000 вычислений:

1. **подход к ограничению**: штраф против ремонта (равномерный кроссовер у обоих);
2. **пара операторов**: равномерный против одноточечного кроссовера (в режиме ремонта — победителя первого эксперимента);
3. **базовые линии**: лучшая конфигурация ГА против жадной эвристики LPT (заявки по убыванию w_i назначаются допустимому исполнителю с наименьшей текущей длительностью) и случайного допустимого поиска (равномерные назначения с ремонтом, бюджет тот же).

Метрики серии: лучшее, среднее, медиана, стандартное отклонение, худшее значение; средние кривые сходимости с коридором мин–макс.

8 Результаты: штраф против ремонта

Таблица 3 – Итоговый makespan по 30 запускам при двух подходах к ограничению

Подход	Лучшее	Среднее	Медиана	Стд	Худшее
штраф	27,08	28,90	28,68	1,16	31,06
ремонт	**26,80**	**27,17**	27,18	0,20	27,48



Рис. 2 – Сходимость при двух подходах к ограничению

Ремонт устойчиво точнее штрафа (среднее 27,17 против 28,90, таблица 3, рис. 2). При штрафе часть популяции постоянно живёт с нарушениями: выигрыш в makespan у сильных исполнителей часто больше штрафа $W = 50$, поэтому отбор ведёт решения в зону нарушений (рис. 3). Ремонт возвращает всех в допустимое множество, и отбор работает по чистому makespan.



Рис. 3 – Штрафной режим: доля допустимых особей в популяции по поколениям

9 Результаты: сравнение кроссоверов

Таблица 4 – Равномерный против одноточечного кроссовера (ремонт, 30 запусков)

Кроссовер	Лучшее	Среднее	Медиана	Стд	Худшее
равномерный	26,80	27,17	27,18	0,20	27,48
одноточечный	**26,72**	**27,00**	**27,00**	0,17	27,41

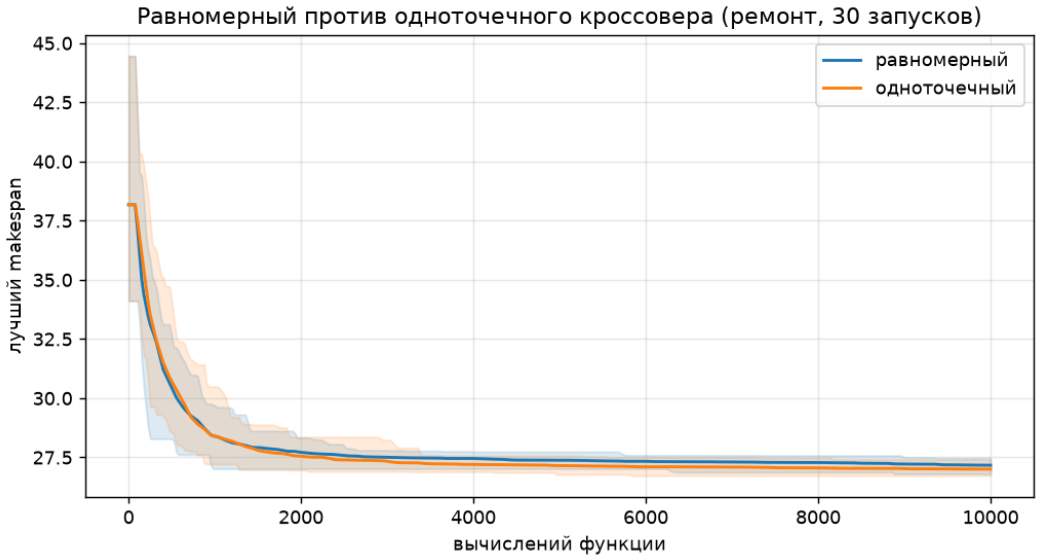


Рис. 4 – Сходимость при двух операторах рекомбинации (режим ремонта)

Одноточечный кроссовер немного лучше равномерного: разрыв 27,00 ↔ 27,17 (таблица 4, рис. 4) невелик по абсолютной величине, но воспроизводится на всех 30 запусках. Равномерный кроссовер дробит решения слишком мелко; одноточечный сохраняет пригодные участки ответов.

10 Результаты: сравнение с базовыми линиями

Таблица 5 – ГА (одноточечный + ремонт) против жадной эвристики и случайного допустимого поиска

Метод	makespan (среднее)
ГА (одноточечный + ремонт), 30 запусков	**27,00**
случайный допустимый поиск (бюджет 10^4), 30 запусков	30,48
жадная эвристика LPT	33,69

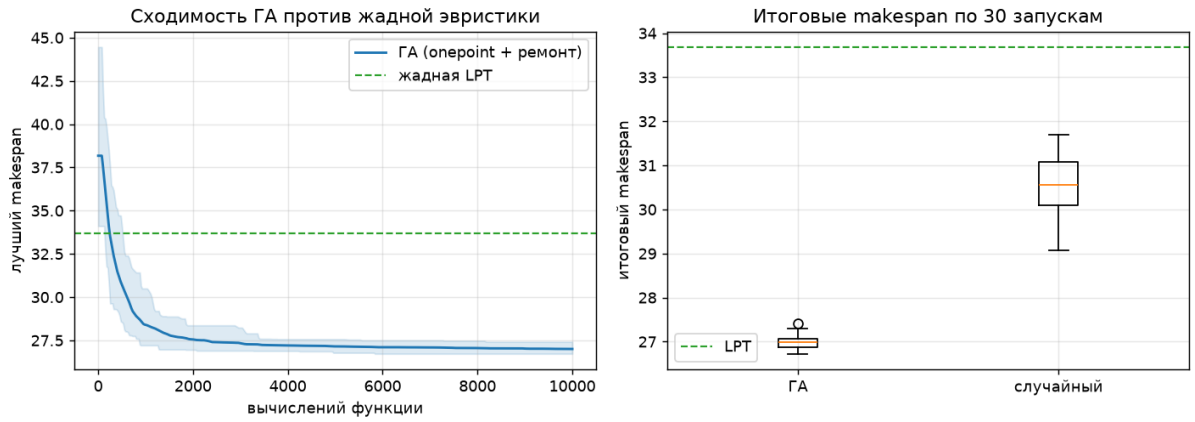


Рис. 5 – Сходимость ГА против жадной эвристики (слева) и итоговые makespan серий (справа)

ГА устойчиво лучше обеих базовых линий (таблица 5, рис. 5): разрыв с жадной LPT — около $\frac{33{,}69-27{,}00}{33{,}69} \approx 20\%$; случайный допустимый поиск без селекции и рекомбинации улучшает результат только за счёт плотности выборки, поэтому и проигрывает (лучшее значение 29,06).

Лучшее решение серии. x^* с makespan 26,724; допустимость по каждому исполнителю соблюдена (таблица 6).

Таблица 6 – Лучшее решение по исполнителям (30 запусков ГА, best)

Исполнитель	Производительность	Ёмкость	Назначено	Длительность (ч)
0	0,5	5	3	26,684
1	0,7	6	4	26,441
2	1,0	7	6	26,507
3	1,3	6	6	26,675
4	1,6	5	5	26,541
5	2,0	6	6	26,724

11 Оценка результата и выводы

Полученный результат — **хороший** для поставленной задачи: все запуски в режиме ремонта дают допустимые решения по построению; лучшая конфигурация (одноточечный кроссовер + ремонт) во всех 30 запусках находит makespan $\leq 27{,}41$, что лучше жадной эвристики примерно на 20%, а случайного допустимого поиска — примерно на 12%; разрыв устойчив по всей серии, а не обязан единичному удачному запуску.

Выводы:

1. ремонт устойчиво точнее штрафа (27,17 против 28,90): для ограничения ёмкости явное восстановление допустимости выигрывает у оценивания штрафом;

2. одноточечный кроссовер немного точнее равномерного в этой задаче (27,00 против 27,17), и разница стабильна при фиксации всех прочих параметров;

3. ГА устойчиво улучшает жадную эвристику LPT (33,69 $\rightarrow$ 27,00) и случайный допустимый поиск (30,48 $\rightarrow$ 27,00);

4. лучшее значение makespan по всей серии — 26,72 при полной допустимости решения.

Состав материалов

- `lab2.ipynb` — ноутбук с полным кодом и всеми экспериментами;
- `report.tex`, `report.pdf` — настоящий отчёт;
- `figures/convergence_constraints.png` — штраф против ремонта;
- `figures/feasibility_penalty.png` — доля допустимых при штрафе;
- `figures/convergence_crossover.png` — сравнение кроссоверов;
- `figures/ga_vs_baselines.png` — ГА против базовых линий;
- `results/runs_constraints.csv` — запуски эксперимента 1;
- `results/runs_crossover.csv` — запуски эксперимента 2;
- `results/runs_baselines.csv` — запуски случайного поиска;
- `results/summary_constraints.csv`, `results/summary_crossover.csv` — сводные статистики;
- `results/best_solution.csv` — лучшее найденное назначение;
- `README.md` — инструкция по запуску и воспроизводимости.